APPLIED AGENTIC AI FOR PMS/TPMS · CAPSTONE

Mira

A multi-agent assistant that turns a project's own files into plans, risk assessments, status reports and stakeholder updates — grounded, measured and traced.

Mallika Annam · November 2026

n8n Cloud · OpenAI gpt-4o-mini · Langfuse

github.com/m-annam/mira-capstone

THE PROBLEM AT NEXORA

A small PM/TPM team loses days a month to documents

3–4 h

to build one project plan from a high-level description

1–2 h

every week to compile a status report from task boards and notes

Inconsistent risks

Some projects have a risk matrix, others none

Missed milestones

No early warning; issues surface in retrospectives

No standard format

Every PM writes plans and reports differently

Source: Mira problem statement (Nexora Pvt. Ltd., client ABCDE Ltd.)

WHAT MIRA DOES

Five specialist agents draft the documents PMs spend hours on

Planner

Project plans

8 phases with dates, milestones and deliverables; goals quoted exactly

Risk Assessor

Risk assessments

Full matrix from the register; top risks ranked by live task evidence

Status Reporter

Status and tracking

Weekly or sprint reports; blocked, overdue and due-soon tasks; milestones

Timeline Advisor

Schedule answers

Current phase and week, weeks left, phase dates and upcoming milestones

Stakeholder Update

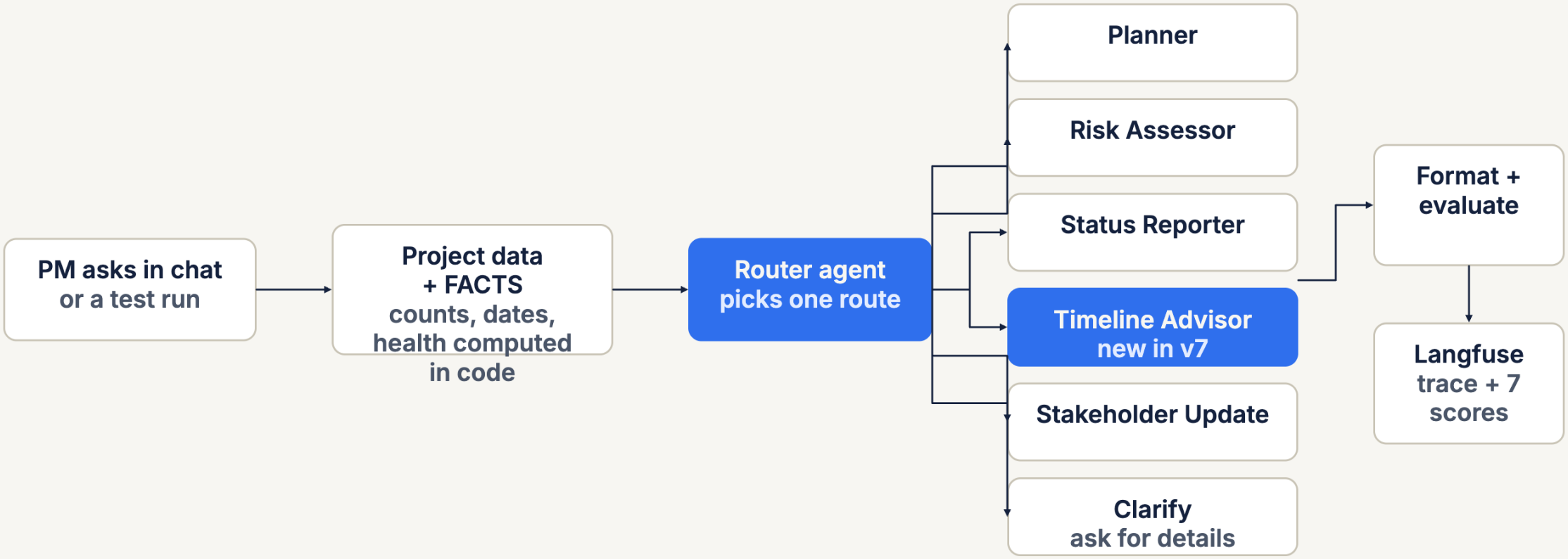
Stakeholder emails

Leadership, engineering or client versions; no owner names for the client

Vague request? Mira asks instead of guessing. Every answer cites its sources and is a “Draft for PM review”.

ARCHITECTURE

A router sends each request to one specialist



n8n Cloud · gpt-4o-mini · Langfuse (OpenTelemetry)

HOW I BUILT IT

Ground truth first, prompts second, then measure

1

Requirements

BRD with baselines and targets; PRD with the router design

2

Ground truth

23 tests, 151 checks, expected values recomputed from the data

3

Facts in code

Counts, overdue tasks, health and dates computed before the model runs

4

Build and trace

n8n workflow from versioned source; every request traced in Langfuse

5

Test and read

Every test as a chat message, scored live; every answer read by hand

6

Repeat to release

Promote a prompt only after three identical runs pass

Prompt versions v1 → v6 · all on the same 23 tests

KEY DECISIONS

Trade-offs I can defend

Decision	Why	What I gave up
Router, not a pipeline	Only the needed agent runs; a clarifying reply costs $0.00019, a plan $0.00077	Two-part requests get only the first part
Facts computed in code	Models miscount and invent; code doesn't	Every new fact needs a function
Data scoped per agent	Less to invent from, less personal data sent	Cross-cutting questions need routing rules
gpt-4o-mini	Good enough once facts come from code	Long answers take 10–12 s
PM reviews every draft	A wrong fact must never reach a client	No full automation in the pilot

MEASURED RESULTS

75 of 75 test requests passed across three identical runs

75/75
tests passed (prompt v7, 3 runs × 25 tests); target ≥ 11 of 12 baseline

100%
routed to the right agent; target ≥ 95%

0
invented tasks, risks or milestones in 75 answers

$0.00065
per request, from real token counts; about $0.07 per PM per month

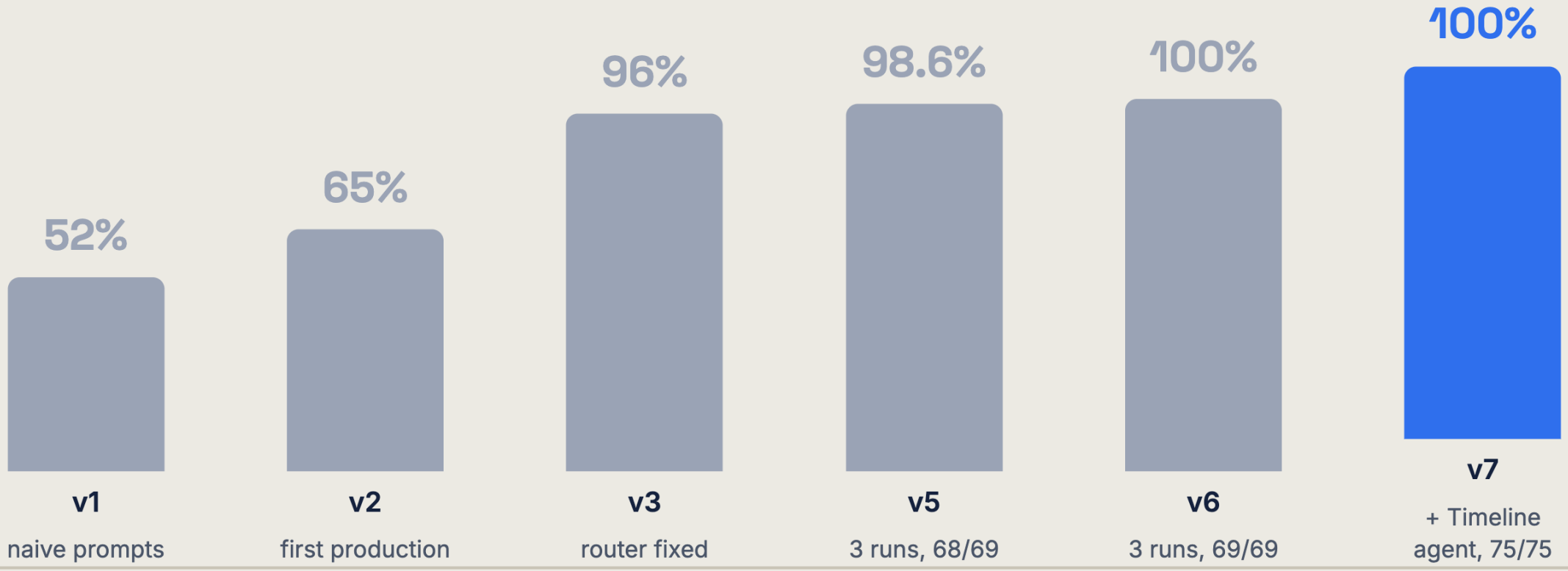
3–5 s
median response (3.2 s in v6, 4.9 s on a slower API day in v7); long plans and risk matrices take longer

~11 h
PM time saved per month (planning estimate, to measure in the pilot)

Live on n8n Cloud, executions #234–#309, traced in Langfuse

PROMPT EXPERIMENT

Same data, same tests: pass rate rose from 52% to 100%



All-tests pass rate per prompt version; v4 was graded on tightened checks (23/23); v7 adds 2 timeline tests (25)

WHAT BROKE, AND HOW I FIXED IT

The traces found three problems the scores hid

Routing fell to 74%

The router asked questions it could answer

It never sees the data, so “no data → ask” fired on every status question. Telling it the specialists hold all the data fixed it: 100%.

12/12 that wasn’t

Two wrong answers passed their checks

One count list dropped “Blocked”; a sprint report used project-wide health. Tighter checks, a new prompt, then per-sprint facts in code.

1 run in 3

One passing run was not evidence

Three runs showed the plan dropping phase names once. A one-line rule fixed it, and promotion now needs three clean runs.

A green test suite is only as good as its assertions.

ROLLOUT TO 10 PMS/TPMS

Five weeks from sign-off to a go / no-go decision

Week 0

Readiness

Security sign-off, EU tracing region, 30-minute training

Weeks 1–2

Champions

3 PMs on ABCDE. Gate: ≥ 70% of drafts sent with minor or no edits

Weeks 3–4

Pilot

All 10 PMs, 2+ projects. Gate: 8 of 10 use it weekly

Week 5

Decision

CTO decides go, adjust or stop against the BRD targets

Next: CSV upload in chat, milestone alerts in the Monday email, a router for two-part requests, and an AI judge for tone and completeness.

Program charter: docs/10-program-charter.md

LESSONS

Put facts in code, judgement in prompts — and read the outputs

Ground truth first

Asking “how will this be checked?” shaped the prompts

Read every answer

Scores said 12/12; reading found two wrong answers

One run is not evidence

Promote a version only after repeated clean runs

Repo, docs, screenshots and results: github.com/m-annam/mira-capstone